

DIVIDE AND CONQUER

- Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

CODE:

```
#include <stdio.h>
```

```
int firstZero(int a[],int low,int high){
```

```
if(low>high)
```

```
return -1;
```

```
int mid=(low+high)/2;
```

```
if(a[mid]==0 && (mid==0 || a[mid-1]==1))
```

```
return mid;
```

```
if(a[mid]==1)
```

```
return firstZero(a,mid+1,high);
```

```
return firstZero(a,low,mid-1);
```

```
}
```

```
int main(){
```

```
int m;
```

```

scanf("%d",&m);

int a[m];

for(int i=0;i<m;i++)

scanf("%d",&a[i]);

int index=firstZero(a,0,m-1);

if(index==-1)

printf("0\n");

else

printf("%d\n",m-index);

return 0;

}

```

- Given an array nums of size n , return the majority element .

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 \cdot 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int nums[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &nums[i]);
```

```
    }
```

```
    int candidate = 0;
```

```
    int count = 0;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (count == 0) {
```

```
            candidate = nums[i];
```

```
        }
```

```
        if (nums[i] == candidate) {
```

```
            count++;
```

```

} else {

count--;

}

}

printf("%d\n", candidate);

return 0;

}

```

- Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

CODE:

```
#include <stdio.h>
```

```
int findFloor(int a[], int low, int high, int x) {
```

```
if (low > high)
```

```
return -1;
```

```
int mid = low + (high - low) / 2;
```

```
if (a[mid] == x)
```

```

return a[mid];

if (a[mid] > x)

return findFloor(a, low, mid - 1, x);

int result = findFloor(a, mid + 1, high, x);

if (result == -1)

return a[mid];

return result;

}

int main() {

int n, x;

scanf("%d", &n);

int a[n];

for (int i = 0; i < n; i++)

scanf("%d", &a[i]);

scanf("%d", &x);

printf("%d\n", findFloor(a, 0, n - 1, x));

return 0;

}

```

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using

divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

CODE:

```
#include <stdio.h>
```

```
int findPair(int arr[], int left, int right, int x) {
```

```
    if (left >= right)
```

```
        return 0;
```

```
    int sum = arr[left] + arr[right];
```

```
    if (sum == x) {
```

```
        printf("%d\n", arr[left]);
```

```
        printf("%d\n", arr[right]);
```

```
        return 1;
```

```
    }
```

```
    if (sum < x)
```

```
        return findPair(arr, left + 1, right, x);
```

```
else
```

```
return findPair(arr, left, right - 1, x);
```

```
}
```

```
int main() {
```

```
int n, x;
```

```
scanf("%d", &n);
```

```
int arr[n];
```

```
for (int i = 0; i < n; i++)
```

```
scanf("%d", &arr[i]);
```

```
scanf("%d", &x);
```

```
if (!findPair(arr, 0, n - 1, x))
```

```
printf("No\n");
```

```
return 0;
```

```
}
```

- Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input

Result

5

67 34 12 98 78 12 34 67 78 98

CODE:

```
#include <stdio.h>
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pivot = arr[high];
```

```
        int i = low - 1;
```

```
        int temp;
```

```
        for (int j = low; j < high; j++) {
```

```
            if (arr[j] <= pivot) {
```

```
                i++;
```

```
                temp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = temp;
```

```
            }
```

```
        }
```

```
        temp = arr[i + 1];
```

```
arr[i + 1] = arr[high];
```

```
arr[high] = temp;
```

```
int pi = i + 1;
```

```
quickSort(arr, low, pi - 1);
```

```
quickSort(arr, pi + 1, high);
```

```
}
```

```
}
```

```
int main() {
```

```
int n;
```

```
scanf("%d", &n);
```

```
int arr[n];
```

```
for (int i = 0; i < n; i++) {
```

```
scanf("%d", &arr[i]);
```

```
}
```

```
quickSort(arr, 0, n - 1);
```

```
for (int i = 0; i < n; i++) {
```

```
if (i > 0)
```

```
printf(" ");
```

```
printf("%d", arr[i]);
```

```
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```